UNIVERSIDADE FEDERAL DO CEARÁ
MESTRADO E DOUTORADO EM CIÊNCIA DA COMPUTAÇÃO (MDCC)
CKP7500 – SISTEMAS DISTRIBUÍDOS E REDES DE COMUNICAÇÃO
Prof. Windson Viana de Carvalho Prof. Fernando Antonio Mota Trinta

Invocação Remota e Comunicação Indireta: uma comparação experimental entre Socket, RMI, JSON-RPC, HTTP/REST e MQTT

Lucas Alves Meneses (603270)
Beloin Sena de Farias Rodrigues (603022)

Atividade AT03 • Fortaleza, 2026

Resumo

Este relatório compara cinco modelos de comunicação a partir de uma calculadora remota implementada com Socket, RMI, JSON-RPC e HTTP/REST e de um sistema de monitoramento de caldeira com MQTT. O tráfego de cada implementação foi capturado no Wireshark executando as mesmas operações. As capturas mostram que cada modelo concentra seu custo em um ponto diferente: segmentação no Socket, controle do middleware no RMI, cabeçalhos HTTP no JSON-RPC e no REST, e multiplicação de mensagens pelo broker no MQTT, que, em troca, é o único a oferecer desacoplamento espacial entre os componentes.

Palavras-chave: sistemas distribuídos. invocação remota. publish-subscribe. MQTT. Wireshark.

1 Introdução

A atividade consistiu em evoluir os projetos de calculadora remota da disciplina para suportar as quatro operações básicas e expressões com parênteses, como $(10 + 15) \times 4$, em duas abordagens: expressão *decomposta no cliente*, que invoca uma operação remota para cada operador, e expressão *calculada no servidor*, enviada inteira em uma única chamada. O cliente HTTP implementa uma política de retry. Além disso, foi construído um sistema MQTT de monitoramento industrial com sensores simulados, um sensor em dispositivo real e os serviços CAT e Alarms.

Todo o código está no repositório `https://github.com/Beloin/PostGradRemoteInvocation`, e cada projeto é executado por um alvo do `Makefile` (Tabela 1). O vídeo de demonstração do MQTT está em `https://drive.google.com/file/d/1cH7crceAA0Ld_9H1FBfPYO8gDzq0V7cE/view`.

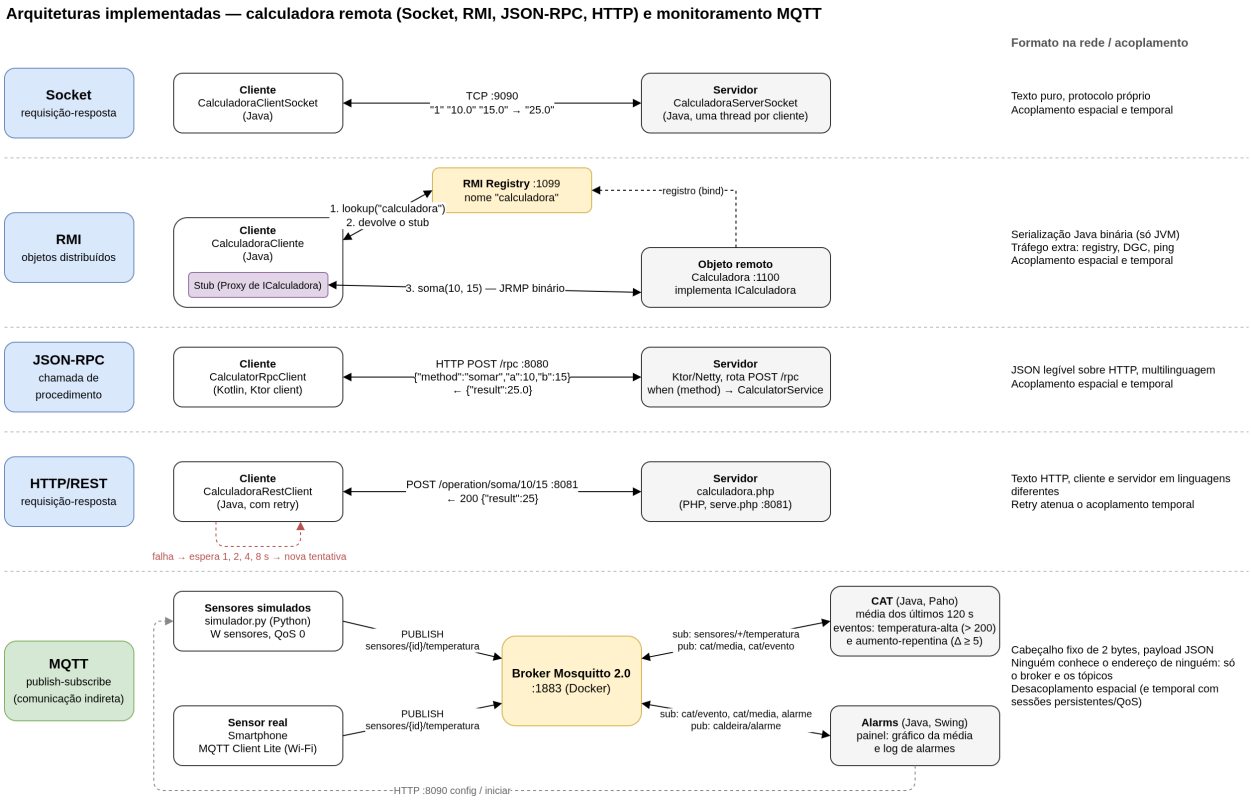
Tabela 1 – Projetos, tecnologias e comandos de execução

Modelo	Pasta e tecnologia	Execução
Socket	`JavaClientSocket_Calculadora`, `JavaServerSocket_Calculadora` (Java)	`make run-socket`
RMI	`ExemploRMI` (Java RMI)	`make run-rmi`
JSON-RPC	`rpcKTOR` (Kotlin, Ktor/Netty)	`make run-rpc`
HTTP/REST	`ClientHTTPSCalculadora` (Java/Swing) + `calculadoraPHP` (PHP)	`make server-php`, `client-http`
MQTT	`MQTTMonitor`: CAT e Alarms (Java, Paho), `sensorSimulator` (Python), Mosquitto 2.0 (Docker); sensor real: smartphone com o app MQTT Client Lite	`make mqtt-up`, `cat-service`, `alarms`, `sensor-sim`

2 Arquiteturas implementadas

A Figura 1 resume as cinco arquiteturas. As calculadoras compartilham o mesmo avaliador de expressões (`Avaliador`), que trata parênteses e precedência: no modo decomposto ele roda no cliente; no modo servidor, do lado do servidor.

Figura 1 – Arquiteturas implementadas e seus formatos de comunicação



Fonte: elaborado pelos autores.

Socket. O servidor abre um `ServerSocket` na porta 9090 e atende cada cliente em uma thread. O cliente escreve linhas de texto com o código da operação (`1` a `4`, ou `expr`) e os operandos; todo o protocolo foi definido pelos próprios programas.

RMI. A interface `ICalculadora` estende `java.rmi.Remote`. O servidor exporta o objeto na porta 1100 e o registra como `calculadora` no registry da porta 1099; o cliente faz `lookup`, recebe um stub e invoca os métodos como se fossem locais.

JSON-RPC. O servidor Ktor expõe `POST /rpc` na porta 8080; o corpo JSON traz o nome do método e os argumentos, despachados por um `when (req.method)` para o `CalculatorService`.

HTTP/REST. O cliente envia `POST /operation/{op}/{a}/{b}` à calculadora PHP na porta 8081 e recebe `{"result": ...}`. Em caso de falha, repete a requisição com *exponential backoff* (1, 2, 4 e 8 s, até quatro novas tentativas), fora da thread da interface. Conforme o roteiro, este projeto implementa apenas as quatro operações.

MQTT. Sensores publicam em `caldeira/sensores/{id}/temperatura`. O CAT assina o tópico `caldeira/sensores/+/temperatura` — o curinga `+` permite adicionar ou remover sensores sem alterar o CAT —, mantém as leituras dos últimos 120 s e publica a média e os eventos `temperatura-alta` (média acima de 200 °C) e `aumento-repentina` (duas médias consecutivas com diferença de 5 °C ou mais). O Alarms assina os eventos e os exibe em uma interface Swing. O sensor real é um smartphone com o aplicativo MQTT Client Lite, no qual os valores de temperatura são informados manualmente e publicados no mesmo padrão de tópico e payload. Nenhum componente conhece o endereço dos outros, apenas o do broker.

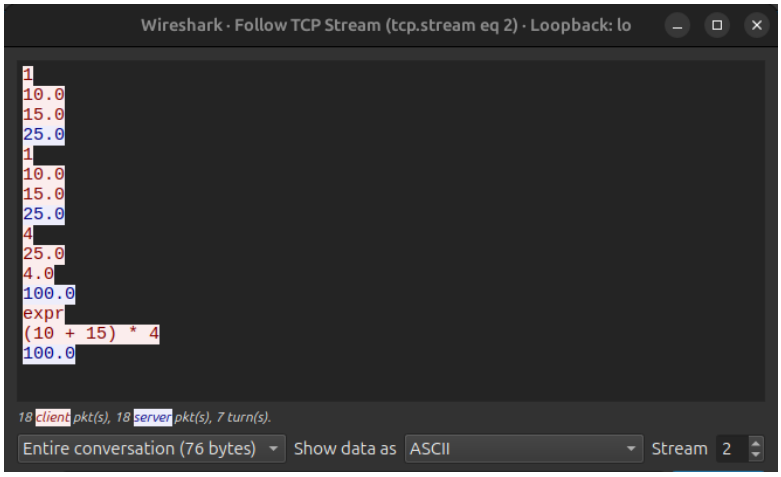
3 Metodologia de captura

As capturas foram feitas no Wireshark 4.2.2, em Ubuntu 24.04.5 com OpenJDK 17 e PHP 8.3, com clientes e servidores na mesma máquina (interface `lo`; para o MQTT, interface `any`, pois o broker roda em contêiner Docker). Em cada calculadora foi executado o mesmo roteiro: *(i)* $10 + 15$; *(ii)* $(10 + 15) \times 4$ decomposta no cliente; *(iii)* a mesma expressão calculada no servidor. As métricas vêm de *Statistics* → *Conversations* e o conteúdo, de *Follow TCP/HTTP Stream*. Os arquivos `.pcapng` acompanham o trabalho.

4 Análise das capturas

4.1 Socket

Figura 2 – Socket: conteúdo da conexão na porta 9090 (cliente em vermelho, servidor em azul)

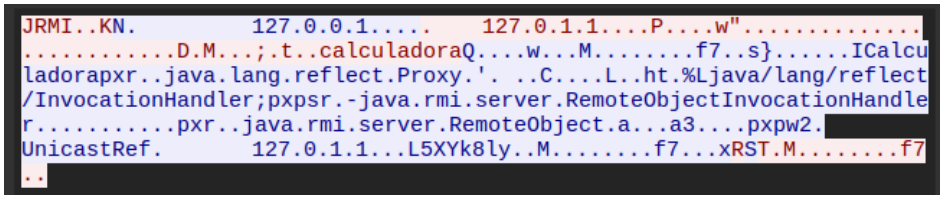


Fonte: elaborado pelos autores.

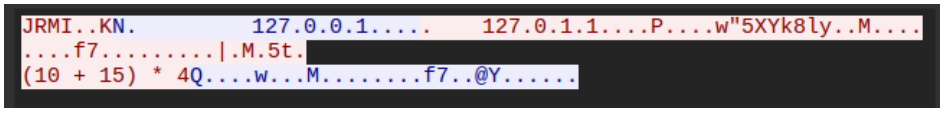
O socket transporta texto puro (Figura 2): cada requisição são três linhas (operação e operandos) e a resposta é uma linha. Na expressão decomposta aparecem duas trocas (`1 10.0 15.0` → `25.0` e `4 25.0 4.0` → `100.0`); na calculada no servidor, uma só (`expr` e a expressão). O significado do código `4` (multiplicação) existe apenas no acordo entre os dois programas. Os três testes usaram uma conexão e 78 pacotes para apenas 76 bytes de dados: o cliente usa `DataOutputStream.writeBytes` sem buffer, e o TCP enviou cada caractere em um segmento — a primeira soma, de 17 bytes, gerou 34 pacotes.

4.2 RMI

Figura 3 – RMI: (a) lookup no registry, que devolve o stub; (b) chamada de `avaliar` na porta 1100



(a) conexão com o registry (porta 1099)



(b) chamada remota da expressão (porta 1100)

Fonte: elaborado pelos autores.

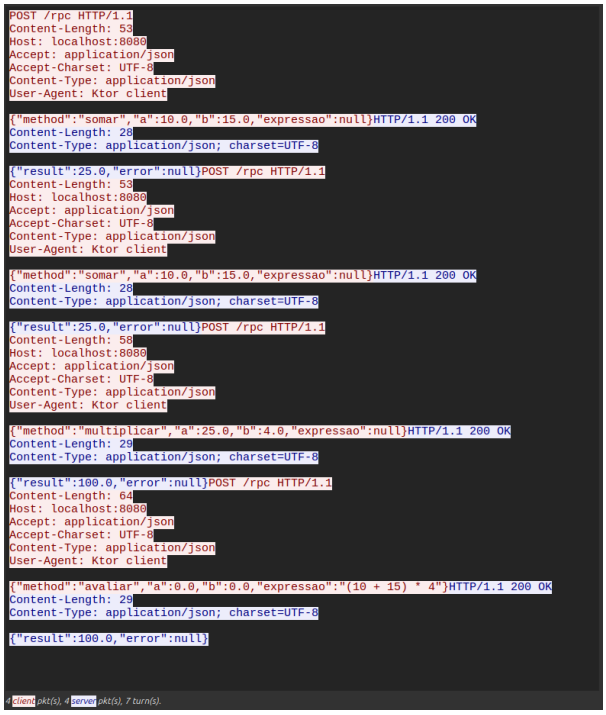
Toda conexão começa com a assinatura `JRMI` do protocolo JRMP. No lookup (Figura 3a), o cliente pede `calculadora` e o registry devolve o stub serializado: um `java.lang.reflect.Proxy` de `ICalculadora` com uma `UnicastRef` que contém o endereço e a porta 1100 (bytes `04 4C`) do objeto. O cliente nunca configurou essa porta: ela chega dentro do stub, o que é a transparência de localização do RMI. Na chamada (Figura 3b), a string `(10 + 15) * 4` aparece legível, mas o resultado volta como `@Y` seguido de seis bytes nulos, o `double` 100.0 em IEEE 754 (`0x4059000000000000`). Nas operações básicas os argumentos vão como `int` de 4 bytes, e o método é identificado por um hash de 8 bytes, não pelo nome.

A captura registrou cinco conexões: lookup no registry (18 pacotes), coletor de lixo distribuído com `Lease` e `VMID` (15), e uma por teste (15, 17 e 15). Ou seja, 33 dos 80 pacotes foram controle do middleware antes de qualquer cálculo, e cada teste reabriu a conexão porque o RMI fecha conexões ociosas após cerca de 15 s.

4.3 JSON-RPC

No JSON-RPC tudo é legível (Figura 4): o corpo traz o nome do método (`"somar"`, `"multiplicar"`, `"avaliar"`) e a resposta, `result` e `error`. Os três testes usaram uma única conexão mantida por keep-alive (25 pacotes), mas 75% dos 1366 bytes de dados foram cabeçalho HTTP: 170 bytes por requisição para corpos de 53 a 64 bytes. A implementação segue a ideia do JSON-RPC, mas não o formato da especificação 2.0, que exige os campos `jsonrpc`, `params` e `id`.

Figura 4 – JSON-RPC: as quatro chamadas `POST /rpc` e suas respostas na mesma conexão



Fonte: elaborado pelos autores.

4.4 HTTP/REST e política de retry

Uma requisição bem-sucedida (`POST /operation/soma/10/15`, resposta `{"result":25}`) tem 165 bytes de linha de requisição e cabeçalho, sem corpo, e uma resposta de 154 bytes de cabeçalho para 13 de corpo. Embora o cliente peça `keep-alive`, o servidor embutido do PHP responde `Connection: close`, e cada requisição abre sua própria conexão TCP (12 pacotes).

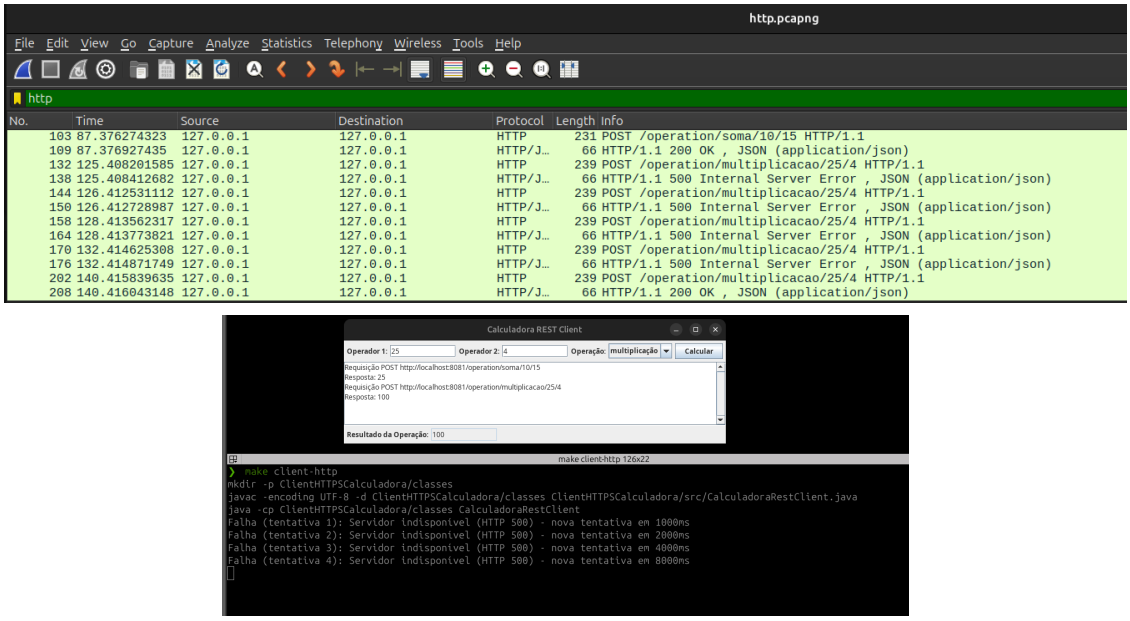
Para testar o retry, o servidor foi bloqueado antes de uma multiplicação e liberado durante as tentativas (Figura 5). Os instantes das requisições — 125,408; 126,413; 128,414; 132,415 e 140,416 s — mostram esperas de exatamente 1, 2, 4 e 8 s: quatro respostas `500` e sucesso na quinta tentativa, a última permitida. O retry é seguro aqui porque a operação é idempotente; em uma operação com efeito colateral, uma tentativa cuja resposta se perdeu poderia executá-la duas vezes (semântica *at-least-once* [1]).

4.5 MQTT

Para acelerar a demonstração, o simulador usou 3 sensores, intervalo de 5 s e temperatura base de 195 °C. Cada serviço abriu uma única conexão persistente com o broker (Figura 6): o CAT envia `CONNECT` com keep-alive de 60 s e assina o tópico com QoS 1; o Alarms assina três tópicos; o simulador apenas conecta. Sem tráfego, os clientes trocam `PINGREQ`/`PINGRESP` de 2 bytes.

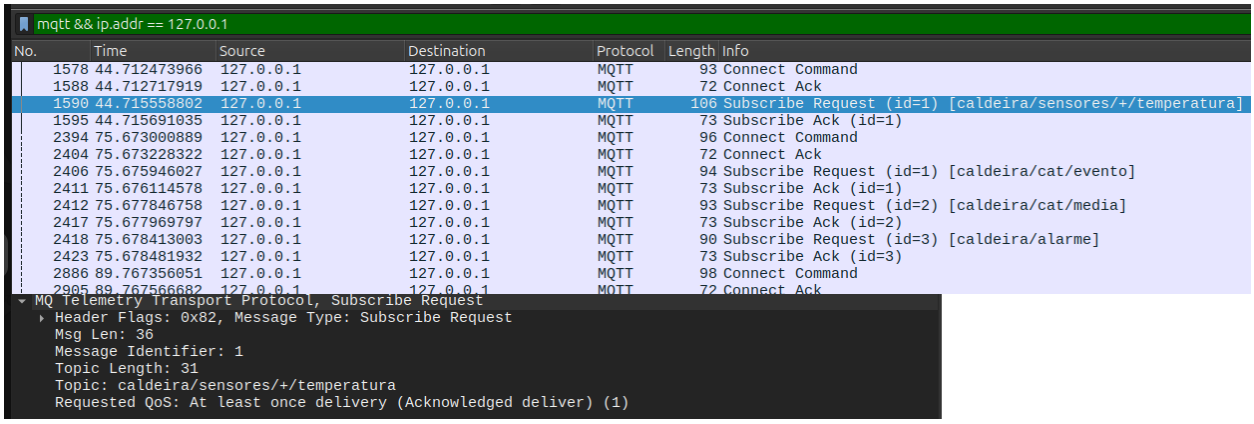
Cada leitura é um `PUBLISH` de 101 bytes: 2 de cabeçalho fixo, 33 de tópico e 66 de payload JSON (Figura 7). Os sensores publicam com QoS 0; CAT e Alarms, com QoS 1, e cada mensagem deles recebe um `PUBACK`. Uma única leitura desencadeou 8 `PUBLISH`, pois tudo passa duas vezes pelo broker: sensor → CAT, média e evento do CAT → Alarms, e o alarme do Alarms → ele próprio. Em cerca de 110 s houve 69 leituras, 552 `PUBLISH` e 414 `PUBACK`, todos pela porta 1883. A Figura 8 mostra o painel exibindo os alarmes. A captura revelou que o CAT publica `temperatura-alta` a cada leitura enquanto a média permanece alta; publicar apenas na transição reduziria tráfego e o

Figura 5 – Retry com exponential backoff: requisições na rede e log do cliente



Fonte: elaborado pelos autores.

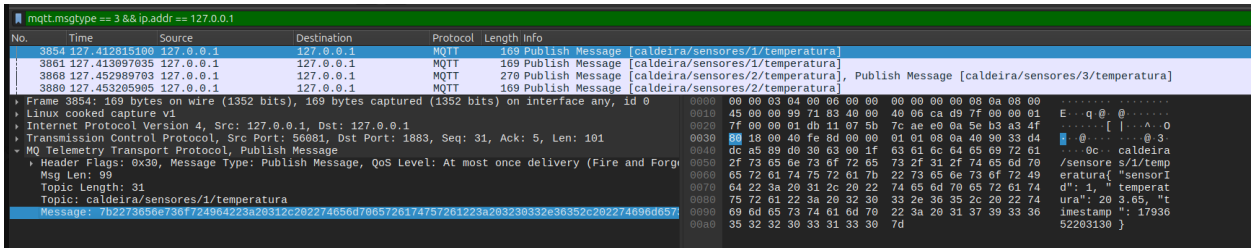
Figura 6 – MQTT: conexões e assinaturas, com o detalhe do SUBSCRIBE do CAT



Fonte: elaborado pelos autores.

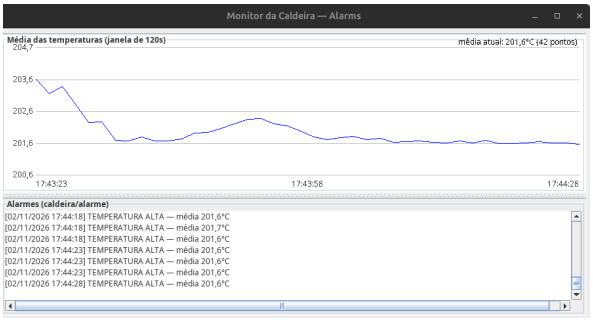
excesso de alarmes.

Figura 7 – MQTT: PUBLISH de uma leitura de temperatura, com tópico, QoS 0 e payload



Fonte: elaborado pelos autores.

Figura 8 – Painel do serviço Alarms: média da janela de 120 s e alarmes de temperatura alta



Fonte: elaborado pelos autores.

4.6 Resumo das medições

Tabela 2 – Comparação das capturas

Modelo	Conexões	Pacotes	Formato na rede	Principal overhead
Socket (3 testes)	1	78	Texto, protocolo próprio	Um segmento por caractere
RMI (3 testes)	5	80	Binário (serialização Java)	Registry, DGC, reconexão
JSON-RPC (3 testes)	1	25	JSON sobre HTTP	Cabeçalho HTTP (75% dos dados)
HTTP/REST (1 soma)	1	12	URI + JSON sobre HTTP	Uma conexão por requisição
MQTT (1 leitura)	persistentes	—	2 B de cabeçalho + JSON	8 PUBLISH por leitura

Nas três calculadoras com expressões, a abordagem decomposta exigiu duas chamadas remotas e a calculada no servidor, uma (no RMI, 17 contra 15 pacotes; no JSON-RPC, dois `POST` contra um). A diferença cresce com o número de operadores: em uma rede com dezenas de milissegundos de latência, cada operador a mais custa uma ida e volta completa.

5 Análise comparativa

Interoperabilidade. O RMI é o menos interoperável: exige JVM nos dois lados, a mesma interface compilada e serialização Java, e identifica métodos por um hash gerado pelo middleware. O socket funciona em qualquer linguagem, mas seu formato é privado. JSON-RPC e HTTP/REST usam padrões abertos — o próprio projeto liga um cliente Java a um servidor PHP. No MQTT, componentes em Java, Python e um aplicativo de smartphone cooperam sem código em comum, apenas com o formato do tópico e do payload.

Desacoplamento. Socket, RMI, JSON-RPC e HTTP têm acoplamento espacial (o cliente precisa saber onde está o servidor; no RMI, ao menos o registry) e temporal (ambos ativos ao mesmo tempo, com o cliente bloqueado esperando a resposta). O retry atenua o acoplamento temporal, tolerando quedas de até cerca de 15 s, mas não o elimina. O MQTT oferece desacoplamento espacial — sensores não sabem que o CAT existe, e o CAT não sabe quantos sensores há — e pode oferecer desacoplamento temporal com sessões persistentes, QoS 1 ou 2 e mensagens retidas. Em troca, o broker centralizado é ponto único de falha e possível gargalo [1].

Desempenho. Overhead de protocolo mínimo não garante eficiência: o socket, sem cabeçalho de aplicação, gerou um pacote por caractere. O RMI tem chamadas compactas, mas gastou 41%

dos pacotes em controle do middleware. JSON-RPC e REST trocam bytes por legibilidade, com cabeçalhos que dominam mensagens pequenas, e o REST ainda reabre a conexão a cada requisição. O MQTT tem o menor cabeçalho (2 bytes contra 170 do HTTP) e paga o custo de conexão uma única vez, mas a indireção multiplica as mensagens.

Tabela 3 – Vantagens, desvantagens e cenários adequados

Modelo	Vantagens	Desvantagens	Cenário adequado
Socket	Controle total, sem overhead de protocolo	Protocolo e eficiência a cargo do programador	Protocolos próprios de baixa latência (jogos, streaming)
RMI	Chamada remota transparente, tipagem forte	Só JVM, tráfego de middleware, portas dinâmicas	Sistemas Java internos e fechados
JSON-RPC	Simples, legível, multilinguagem	Overhead HTTP, sem padronização de recursos	Integração entre serviços orientada a ações
HTTP/REST	Padrão da web, ferramentas, proxies e cache	Verboso, síncrono	APIs públicas, web e mobile
MQTT	Leve, um-para-muitos, desacoplado	Depende do broker, sem resposta direta	IoT, telemetria e monitoramento

6 Conclusão

Nenhum modelo é superior em todos os critérios, e as capturas tornaram visíveis as trocas de cada um. O socket dá controle total, mas deixa tudo a cargo do programador, inclusive a eficiência. O RMI é o mais confortável de programar em Java, ao custo de tráfego de middleware e de restringir o sistema à JVM. JSON-RPC e HTTP/REST trocam bytes por interoperabilidade. Para o cenário da caldeira, o MQTT é o mais adequado: sensores publicam sem saber quem consome os dados, serviços podem ser substituídos ou multiplicados sem alterar os sensores, e o cabeçalho mínimo convém a dispositivos limitados. Já quando o cliente precisa de uma resposta imediata a um pedido, como na calculadora, os modelos de requisição-resposta continuam adequados, e reduzir o número de idas e voltas importa mais do que o protocolo escolhido.

Referências

[1] COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

[2] OASIS. **MQTT Version 5.0**. 2019. Disponível em: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[3] JSON-RPC WORKING GROUP. **JSON-RPC 2.0 Specification**. 2013. Disponível em: https://www.jsonrpc.org/specification.

[4] GUBBI, J. et al. Internet of Things (IoT): a vision, architectural elements, and future directions. **Future Generation Computer Systems**, v. 29, n. 7, p. 1645–1660, 2013.